

Graph Management System Documentation

1. Introduction

The **Graph Management System** is a Python-based application designed to handle directed graphs. It provides functionalities for creating, modifying, and analyzing graphs, including operations on vertices and edges. The system includes a **Graph Class** for managing graph structures and a **User Interface (UI) Class** for interacting with users.

2. System Components

2.1. Graph Class

The DirectedGraph class represents a directed graph using adjacency lists for efficient storage and manipulation.

2.1.1. Attributes

- `vertices`: A set containing all vertices in the graph.
- `__in`: A dictionary mapping each vertex to its list of incoming edges.
- `__out`: A dictionary mapping each vertex to its list of outgoing edges.
- `__costs`: A dictionary storing edge costs, where keys are (source, destination) pairs.

2.1.2. Methods

- **Vertex Operations:**
 - `add_vertex(vertex)`: Adds a vertex to the graph.
 - `remove_vertex(vertex)`: Removes a vertex and all its associated edges.
 - `get_number_vertices()`: Returns the number of vertices.
 - `parse_vertices()`: Iterates over all vertices.
- **Edge Operations:**
 - `add_edge(source, destination, cost)`: Adds an edge with a given cost.
 - `remove_edge(source, destination)`: Removes an edge.
 - `is_edge(source, destination)`: Checks if an edge exists.
 - `get_number_edges()`: Returns the total number of edges.
 - `get_edges()`: Returns a dictionary of all edges with costs.
 - `get_cost(edge_id)`: Retrieves the cost of an edge.

- `set_cost(edge_id, cost)`: Updates the cost of an edge.
- **Graph Analysis:**
 - `get_inbound_degree(vertex)`: Returns the in-degree of a vertex.
 - `get_outbound_degree(vertex)`: Returns the out-degree of a vertex.
 - `parse_in(vertex)`: Iterates over inbound edges.
 - `parse_out(vertex)`: Iterates over outbound edges.
- **Graph Utilities:**
 - `DeepCopy()`: Returns a deep copy of the graph.
 - `delete_graph()`: Clears all graph data.

2.2. User Interface (UI) Class

The UI class provides an interactive menu for users to manage graphs.

2.2.1. Methods

- **Graph Creation & Management:**
 - `generate_random_graph(vertices, edges)`: Creates a random graph.
 - `create_random_graph()`: Prompts user for parameters to generate a graph.
 - `copy_graph()`: Creates a deep copy of the graph.
- **Graph Information:**
 - `number_of_vertices()`: Displays the number of vertices.
 - `parse_vertices()`: Lists all vertices.
 - `is_edge()`: Checks if an edge exists.
 - `inbound_degree()`: Displays in-degree of a vertex.
 - `outbound_degree()`: Displays out-degree of a vertex.
- **Graph Modification:**
 - `add_vertex()`, `remove_vertex()`
 - `add_edge()`, `remove_edge()`
 - `get_cost()`, `set_cost()`
- **File Operations:**

- `read_graph()`: Reads a graph from a file.
- `write_graph()`: Writes the graph to a file.
- **Menu System:**
 - `print_menu()`: Displays available commands.
 - `start()`: Runs the user interface loop.

3. File Operations

- `read_graph_from_file(file_path)`: Loads a graph from a file.
- `write_graph_to_file(graph, file_path)`: Saves a graph to a file.

4. Execution

Run the program by executing `main.py`, which starts the UI and allows user interaction.